
Projet Atlas

Portail Helpdesk GLPI — Plan de Reprise d'Activité

Procédure d'exploitation du POC (Infrastructure as Code)

Contraintes de reprise : $RPO \leq 20 \text{ min}$ | $RTO \leq 40 \text{ min}$

Méthode : 100% IaC (Ansible) — aucun « clic-clic » non tracé

Hyperviseur : Proxmox VE 8.4 (pve01, 138.201.135.108)

Périmètre : 9 conteneurs LXC — 3 VLAN (DMZ / LAN-ADMIN / MGMT)

Table des matières

1	Présentation du POC	2
1.1	Objectif	2
1.2	Architecture	2
2	Prérequis et poste de pilotage	2
3	Corrections intégrées à l'IaC (juillet 2026)	3
4	Relancer le playbook — la séquence de tests <i>avant</i> d'appliquer	3
4.1	Étape 1 — Vérification de syntaxe	3
4.2	Étape 2 — Vérifier le périmètre (quoi et où)	4
4.3	Étape 3 — Simulation (dry-run) avec diff	4
4.4	Étape 4 — Application réelle	4
5	PRA — Restauration complète (perte de VM)	5
6	PRA — Restauration granulaire (tickets supprimés)	6
7	Sauvegardes et RPO	6
8	Chaos engineering (validation de la détection)	6
9	Accès aux interfaces (tunnels SSH)	7
10	Aide-mémoire — commandes essentielles	7

1. Présentation du POC

1.1 Objectif

Atlas est un **portail de tickets d'assistance (GLPI)** déployé de bout en bout par Infrastructure as Code. L'objectif du POC est de démontrer qu'un incident majeur (perte de VM, corruption de données, panne de service) peut être détecté puis corrigé **dans les délais contractuels** : $RPO \leq 20$ minutes (perte de données maximale) et $RTO \leq 40$ minutes (durée de remise en service).

1.2 Architecture

Deux chemins réseau *indépendants* coexistent :

- **Chemin utilisateur (web)** : Internet → Reverse Proxy TLS (atlrp01p, DMZ) → GLPI (atlapp01p) → MariaDB (atldb01p).
- **Chemin administration (SSH)** : Admin → Bastion (atlbst01p, MGMT) → rebond vers tous les LXC. Aucun service n'est exposé sur Internet ; l'accès admin passe par tunnels SSH.

Conteneur	VMID	IP	Rôle
atlbst01p	200	10.90.0.10	Bastion SSH — point d'entrée admin unique
atlrp01p	201	10.20.0.10	Reverse proxy Nginx (TLS)
atlapp01p	202	10.20.0.11	Application GLPI (PHP-FPM)
atldb01p	203	10.30.0.10	Base de données MariaDB
atlsmt01p	204	10.30.0.11	Relais SMTP Postfix
atlbxb01p	205	10.30.0.12	Supervision Zabbix (serveur + frontend)
atlbkp01p	206	10.30.0.13	Serveur de sauvegarde (copies distantes)
atlgf01p	207	10.30.0.14	Grafana (visualisation)
atltst01p	208	10.20.0.20	Machine de test (chaos engineering)

TABLE 1 – Les 9 conteneurs LXC du POC Atlas.

VLAN	Sous-réseau	Usage
20 (DMZ)	10.20.0.0/24	Services exposés indirectement (reverse proxy, GLPI, test)
30 (LAN-ADMIN)	10.30.0.0/24	Données & supervision (BDD, SMTP, Zabbix, backup, Grafana)
90 (MGMT)	10.90.0.0/24	Management — bastion SSH uniquement

TABLE 2 – Segmentation réseau. Le transit inter-VLAN est refusé par défaut (politique iptables FORWARD DROP), seuls les flux de la matrice de moindre privilège sont autorisés.

2. Prérequis et poste de pilotage

Toutes les commandes Ansible se lancent depuis le **control node**, l'hyperviseur **pve01** (138.201.135.108), où réside le dépôt du projet.

```
# Se connecter au control node
ssh root@138.201.135.108
cd /root/atlas           # repertoire du depot Ansible

# Verifier les outils
ansible --version        # Ansible core
ansible-inventory -i develop/hosts.yaml --graph  # voir les groupes/notes
```

Pourquoi cette commande ?

La commande `ansible-inventory -graph` affiche l'arborescence des groupes (`proxmox`, `mgmt`, `dmz`, `lan_admin`) et confirme que l'inventaire est lisible *avant* de lancer quoi que ce soit. C'est le premier filet de sécurité : si un hôte manque ici, le playbook échouera.

Éléments sensibles (hors dépôt Git).

- Clé SSH d'administration : `/root/.ssh/atlas_ed25519` (rebond bastion).
- Mot de passe du coffre Ansible : `.vault_pass` (déchiffre les secrets `vault_*`).
- Configuration : `ansible.cfg` pointe déjà l'inventaire, le `roles_path` et le vault. Aucune option à répéter en ligne de commande en usage normal.

Attention

Reprise du control node. Si `pve01` est lui-même le sinistré, Ansible doit pouvoir tourner depuis un poste de secours : copier `atlas_ed25519` + `.vault_pass` (coffre hors site) puis surcharger la clé sans éditer `ansible.cfg` : `ansible-playbook ... -e ansible_ssh_private_key_file=<chemin>`. Cette bascule doit être testée régulièrement pour garantir le RTO.

3. Corrections intégrées à l'IaC (juillet 2026)

Trois corrections de supervision ont été appliquées puis **pérennisées dans le playbook** (elles survivent donc à toute réexécution) :

Fix	Correction	Fichier modifié
B	Flux firewall supervision → bastion (VLAN30→90, ports 10050/10051)	<code>roles/proxmox_network/defaults/main.yaml</code>
C	Agent Zabbix accepte les checks passifs locaux (127.0.0.1)	<code>roles/zabbix_agent/templates/zabbix_agent2.conf.j2</code>
A	Désactivation des items internes « not supported » (bruit)	<code>roles/zabbix/tasks/tune_health.yaml</code>

TABLE 3 – Corrections encodées dans l'IaC. Un simple `-tags proxmox_network,zabbix` redéploie l'ensemble.

4. Relancer le playbook — la séquence de tests *avant* d'appliquer**Astuce**

Règle d'or : on ne lance jamais un `ansible-playbook` « à sec » en production sans avoir passé les 3 filets ci-dessous, dans l'ordre : **(1)** syntaxe, **(2)** périmètre, **(3)** simulation. Puis seulement l'application réelle.

4.1 Étape 1 — Vérification de syntaxe

```
ansible-playbook site.yaml --syntax-check
```

Pourquoi cette commande ?

Analyse le YAML et le graphe des rôles *sans se connecter* à aucune machine. Détecte les fautes d'indentation, les tâches malformées, les rôles introuvables. Coût nul, à lancer après

chaque édition du playbook.

4.2 Étape 2 — Vérifier le périmètre (quoi et où)

```
ansible-playbook site.yaml --list-hosts      # sur quels hotes ca va tourner
ansible-playbook site.yaml --list-tasks      # quelles taches, dans quel ordre
```

Pourquoi cette commande ?

`-list-hosts` garantit qu'on ne va pas toucher un hôte non prévu; `-list-tasks` montre l'enchaînement exact (utile pour repérer où agit un tag). Toujours confirmer la cible *avant* d'exécuter, surtout en production.

4.3 Étape 3 — Simulation (dry-run) avec diff

```
# Simulation globale : rien n'est modifié, on voit ce QUI changerait
ansible-playbook site.yaml --check --diff

# Simulation ciblée sur un seul role + un seul hôte
ansible-playbook site.yaml --check --diff --tags zabbix --limit atlzbx01p
```

Pourquoi cette commande ?

`-check` exécute en « mode blanc » (aucune écriture réelle) et `-diff` affiche les différences ligne à ligne des fichiers qui seraient modifiés. C'est la répétition générale : si le diff montre autre chose que prévu, on corrige *avant* d'appliquer.

Attention

Le mode `-check` n'est pas parfait : les tâches qui dépendent d'un résultat précédent (API, shell) peuvent être « sautées » ou signalées à tort. Il valide l'intention, pas un résultat garanti à 100%.

4.4 Étape 4 — Application réelle

```
# Deploiement complet (idempotent : rejouable sans risque)
ansible-playbook site.yaml

# Application ciblée (recommande pour une correction précise)
ansible-playbook site.yaml --tags zabbix                # supervision seule
ansible-playbook site.yaml --tags proxmox_network       # firewall/VLAN de l'hôte
ansible-playbook site.yaml --tags zabbix --limit atlzbx01p # 1 role + 1 hôte
```

Pourquoi cette commande ?

Le playbook est **idempotent** : le relancer sur une infra déjà conforme ne change rien (état « ok », zéro « changed »). Les **tags** permettent de n'appliquer qu'un sous-ensemble (ex. seulement la supervision), et `-limit` restreint aux hôtes visés : c'est le principe de moindre impact.

Tag	Périmètre
proxmox_network	Réseau VLAN + firewall iptables de l'hyperviseur
proxmox_provision	Création/configuration des conteneurs LXC
common	Socle commun (durcissement SSH, UFW, NTP)
bastion	Bastion SSH
mariadb	Base de données
backup	Chaîne de sauvegarde
glpi / glpi_seed	Application GLPI / données de démo
reverse_proxy	Reverse proxy Nginx TLS
zabbix	Serveur + agents de supervision
grafana	Tableaux de bord
testlab	Machine de chaos engineering

TABLE 4 – Tags disponibles dans `site.yaml` (voir l'en-tête du fichier).

5. PRA — Restauration complète (perte de VM)

Scénario : un ou plusieurs conteneurs sont perdus. Le playbook `pra_restore_full.yaml` reconstruit toute la chaîne *dans l'ordre de dépendance*, restaure les données, puis **mesure le RTO** automatiquement.

```
# 1) Tests prealables (comme en section 4)
ansible-playbook pra_restore_full.yaml --syntax-check
ansible-playbook pra_restore_full.yaml --list-tasks

# 2) Restauration complete (dernier dump distant utilise automatiquement)
ansible-playbook pra_restore_full.yaml

# 3) Restauration a partir d'un dump precis (optionnel)
ansible-playbook pra_restore_full.yaml \
  -e "pra_restore_dump=/var/backups/atlas/db-remote/glpi_dump_20260715_195001.sql.gz"
```

Pourquoi cette commande ?

Sans `pra_restore_dump`, le playbook sélectionne *automatiquement* le dump le plus récent présent sur `atlbkp01p` (`ls -t | head -1`) : on repart donc du point de récupération le plus proche possible (RPO minimal). On ne fournit un dump précis que pour rejouer un état antérieur particulier.

Ce que fait le playbook (résumé).

1. Horodatage de début + journal `/var/log/atlas/pra_restore_full.log`.
2. Reprovisioning LXC, socle `common`, bastion.
3. Reconstruction des services : MariaDB → SMTP → GLPI → reverse proxy → Zabbix → backup.
4. Restauration des données (bloc `block/rescue/always`) : transfert du dump `atlbkp01p` → `atldb01p`, import dans `glpi_db`, restauration du filestore GLPI, puis **suppression du dump temporaire** (jamais de base en clair dans `/tmp`).
5. Vérifications : GLPI en HTTP direct *et* via le reverse proxy HTTPS, comptage des tickets, port Zabbix 10051 ouvert.
6. Calcul et journalisation du **RTO mesuré** (comparé à la cible 40 min).

Attention

En cas d'échec pendant la restauration des données, le bloc `rescue` journalise l'**ÉCHEC** et `any_errors_fatal` interrompt le PRA : on ne poursuit jamais sur une base à moitié restaurée. Le `always` nettoie systématiquement le dump temporaire.

6. PRA — Restauration granulaire (tickets supprimés)

Scénario : des tickets ont été supprimés/corrompus, mais l'infrastructure est saine. On ne veut restaurer *que* les tickets manquants, **sans écraser** ceux créés depuis l'incident.

```
# 1) Choisir le dump antérieur à l'incident (sur atl01p)
ssh root@10.30.0.10 'ls -lht /var/backups/atlas/db/ | head'

# 2) Lancer la restauration granulaire (dump OBLIGATOIRE)
ansible-playbook pra_restore_granular.yaml \
  -e "pra_target_dump=/var/backups/atlas/db/glpi_dump_20260715_193001.sql.gz"
```

Pourquoi cette commande ?

La variable `pra_target_dump` est **obligatoire** (le playbook s'arrête via un `assert` si elle manque) : restaurer des tickets est une opération chirurgicale, on doit choisir explicitement la photo à réinjecter. Aucun automatisme ici, contrairement au restore complet.

Mécanisme de sûreté. Le dump est restauré dans une base *temporaire* (`glpi_restore_tmp`), jamais en production directement. Seules les tables de tickets sont exportées avec `mysqldump -no-create-info -insert-ignore`, puis fusionnées : les `INSERT IGNORE` ne touchent pas les lignes déjà présentes. **Les tickets créés après l'incident ne sont donc jamais écrasés.** La base temporaire est supprimée en fin de course, et le RPO effectif (âge du dump) est journalisé.

7. Sauvegardes et RPO

Les dumps GLPI sont produits **toutes les 15 minutes** (cron 5,20,35,50 * * * *), ce qui garantit un $RPO \leq 15$ min (meilleur que la cible de 20 min). Chaque dump est vérifié (`gzip -t`) puis répliqué hors-site sur `atl01p`.

```
# Verifier les derniers dumps locaux (sur la BDD)
ssh root@10.30.0.10 'ls -lht /var/backups/atlas/db/ | head'

# Verifier la replication hors-site (sur le backup)
ssh root@10.30.0.13 'ls -lht /var/backups/atlas/db-remote/ | head'

# Consulter le journal de dump
ssh root@10.30.0.10 'tail -5 /var/log/atlas/dump_glpi.log'
```

Pourquoi cette commande ?

Vérifier que le dump le plus récent est bien présent *des deux côtés* (local + hors-site) et daté de moins de 15 minutes prouve que la chaîne de sauvegarde est vivante — c'est la condition sine qua non pour tenir le RPO en cas de sinistre.

8. Chaos engineering (validation de la détection)

La machine de lab `atl01p` embarque `chaos.sh` : un générateur d'incidents **contrôlés et réversibles**, détectés par Zabbix. Il sert à prouver que la supervision et les alertes fonctionnent

réellement.

```
# Sur atl1st01p (via rebond bastion) :
/opt/atlas/scripts/chaos.sh run      # un scenario ALEATOIRE (cpu | disk | service)
/opt/atlas/scripts/chaos.sh cpu      # saturation CPU temporaire (stress-ng)
/opt/atlas/scripts/chaos.sh disk     # remplissage disque controle
/opt/atlas/scripts/chaos.sh service  # arret de nginx
/opt/atlas/scripts/chaos.sh clean    # REPARATION : retour a l'etat nominal

# Depuis le control node, en une ligne (ad-hoc Ansible) :
ansible atl1st01p -m command -a "/opt/atlas/scripts/chaos.sh service"
ansible atl1st01p -m command -a "/opt/atlas/scripts/chaos.sh clean"
```

Pourquoi cette commande ?

Déclencher un incident connu permet de vérifier que Zabbix *voit* la panne (alerte), puis que **clean** rétablit l'état nominal. C'est la boucle « je casse → ça alerte → je répare » qui valide la supervision de bout en bout.

Attention

chaos.sh contient un **garde-fou** : il refuse de s'exécuter ailleurs que sur l'hôte de lab attendu (**hostname -s**). Sans cela, une erreur d'inventaire pourrait arrêter Nginx sur le vrai reverse proxy. Un cron déclenche aussi un incident toutes les 2 h, réparé automatiquement 45 min plus tard.

9. Accès aux interfaces (tunnels SSH)

Aucun service n'est exposé sur Internet : on y accède par **tunnel SSH** depuis son poste. Ouvrir le tunnel, puis pointer le navigateur sur **localhost**.

```
# GLPI (via reverse proxy TLS)
ssh -L 8443:10.20.0.10:443 root@138.201.135.108      # -> https://localhost:8443

# Zabbix (supervision)
ssh -L 8080:10.30.0.12:8080 root@138.201.135.108    # -> http://localhost:8080

# Grafana (tableaux de bord)
ssh -L 3000:10.30.0.14:3000 root@138.201.135.108    # -> http://localhost:3000
```

Pourquoi cette commande ?

Le tunnel **ssh -L** crée un pont chiffré entre un port de votre poste et le service interne, sans jamais l'exposer publiquement. La surface d'attaque reste minimale : seul le port SSH de l'hôte est joignable de l'extérieur.

10. Aide-mémoire — commandes essentielles

Objectif	Commande
Vérifier la syntaxe	<code>ansible-playbook site.yaml -syntax-check</code>
Voir hôtes / tâches	<code>ansible-playbook site.yaml -list-hosts / -list-tasks</code>
Simuler (dry-run)	<code>ansible-playbook site.yaml -check -diff</code>
Déployer tout	<code>ansible-playbook site.yaml</code>
Déployer un rôle	<code>ansible-playbook site.yaml -tags zabbix</code>
Restauration complète	<code>ansible-playbook pra_restore_full.yaml</code>
Restauration granulaire	<code>ansible-playbook pra_restore_granular.yaml -e "pra_target_dump=..."</code>
Vérifier les sauvegardes	<code>ssh root@10.30.0.10 'ls -lht /var/backups/atlas/db/'</code>
Lancer un incident chaos	<code>ansible atltst01p -m command -a ".../chaos.sh run"</code>
Réparer le chaos	<code>ansible atltst01p -m command -a ".../chaos.sh clean"</code>

TABLE 5 – Récapitulatif opérationnel.